

Ford–Johnson Sort — Visual Walkthrough

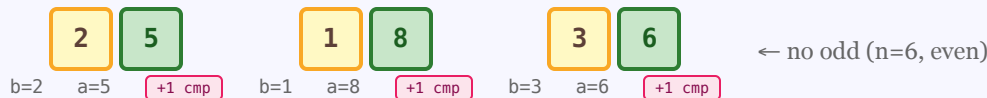
Input: [5, 2, 8, 1, 6, 3] → Expected output: [1, 2, 3, 5, 6, 8]

$n = 6$ | Lower bound $\lceil \log_2(6!) \rceil = 10$ comparisons

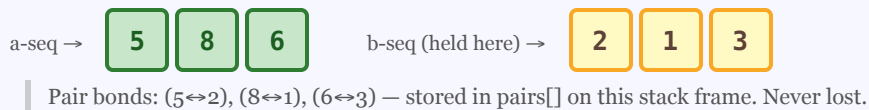
Colours: a a-winner (larger) b b-loser (smaller) ? odd (no partner) b1 free prepend ↑ just inserted

PHASE 2 — DESCENT Depth 0

1 Pair adjacent elements. Compare each pair (1 comparison each).



2 Build a-sequence and recurse on it only. b-elements stay here on the call stack.



↓ recurse([5, 8, 6])

PHASE 2 — DESCENT Depth 1

3 Input is [5, 8, 6]. Pair adjacent. One element left over (odd).



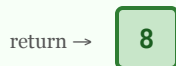
4 Recurse on a-sequence [8] only. Odd element (6) and b-element (5) wait here.



↓ recurse([8])

BASE CASE Depth 2 — size ≤ 1

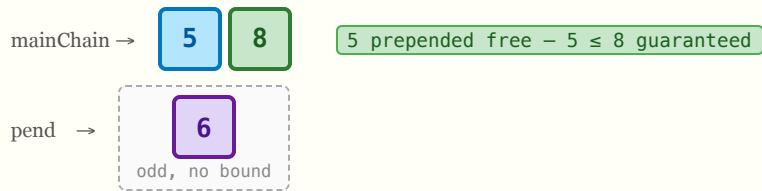
5 Single element [8] is trivially sorted. Return immediately. 0 comparisons.



↑ return [8]

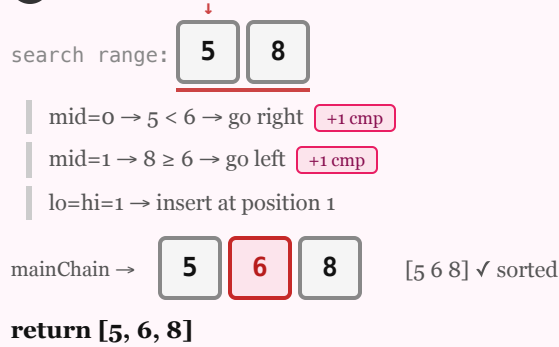
PHASE 3 — UNWIND Depth 1

6 sortedA = [8]. Recover pair: $a=8 \rightarrow b=5$. Prepend $b_1=5$ for free.



PHASE 4 – INSERT Depth 1 – odd element

7 Insert odd 6. No bound $\rightarrow$ search entire chain [5, 8].



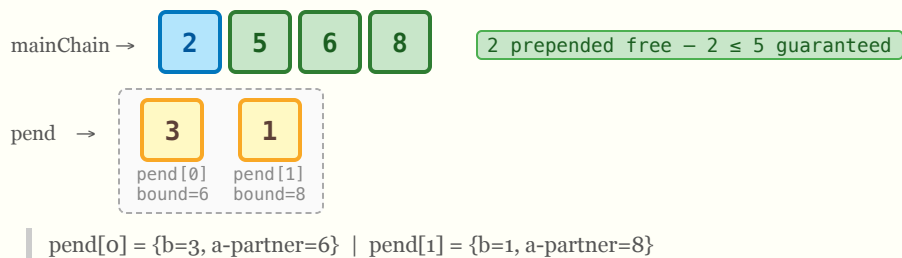
↑ return [5, 6, 8]

PHASE 3 – UNWIND Depth 0

8 sortedA = [5, 6, 8]. Recover pairs from depth-0 stack:

$a=5 \rightarrow b=2$ | $a=6 \rightarrow b=3$ | $a=8 \rightarrow b=1$

Smallest a is 5, so $b_1 = 2$. Prepend free.



PHASE 4 – INSERT Depth 0 – Jacobsthal order

9 Jacobsthal ordering for $m=2$ pend elements:

J sequence: 1, 3, 5, 11, ...

Group 1 covers b-indices $J[1]=3$ down to $J[0]+1=2 \rightarrow b_3, b_2$ In pend terms: $\text{pend}[1] (=b_3)$, then $\text{pend}[0] (=b_2)$ **Order: insert 1 first, then 3**

Why insert 1 before 3? With 1 inserted first, 3's a-partner (6) shifts right by 1. But the Jacobsthal group size is chosen so this shift stays within the same $\lceil \log_2 \rceil$ budget.

10 Insert 1 ($\text{pend}[1]$). a-partner = 8, currently at position 3.findBoundPos: locate 8 in chain $\rightarrow$ index 3. Search range: $[0, 3)$

Chain with search region marked:

mid=1 $\rightarrow 5 \geq 1 \rightarrow$ go left +1 cmpmid=0 $\rightarrow 2 \geq 1 \rightarrow$ go left +1 cmplo=hi=0 $\rightarrow$ insert at position 0mainChain $\rightarrow$ **11 Insert 3** ($\text{pend}[0]$). a-partner = 6, now at position 3 (shifted by earlier insert).findBoundPos: locate 6 in chain $[1,2,5,6,8] \rightarrow$ index 3. Search range: $[0, 3)$

Chain with search region marked:

mid=1 $\rightarrow 2 < 3 \rightarrow$ go right +1 cmpmid=2 $\rightarrow 5 \geq 3 \rightarrow$ go left +1 cmplo=hi=2 $\rightarrow$ insert at position 2mainChain $\rightarrow$ **DONE** Sorted: **[1, 2, 3, 5, 6, 8]** ✓**Comparison tally:**Depth 0 — pairing: $(5,2) + (8,1) + (6,3) = 3$ comparisonsDepth 1 — pairing: $(5,8) = 1$ comparison

Depth 1 — insert odd 6: 2 comparisons

Depth 0 — insert 1: 2 comparisons

Depth 0 — insert 3: 2 comparisons

Total: 10 comparisons = $\lceil \log_2(6!) \rceil = 10$ ✓ optimal

Recursion Tree — Bird's Eye View

```

sort([5, 2, 8, 1, 6, 3])                                depth 0
  ↓ pair: (5↔2) (8↔1) (6↔3)   b=[2,1,3] held on stack
  ↓ recurse on a=[5, 8, 6]

sort([5, 8, 6])                                          depth 1
  ↓ pair: (5↔8)   odd=6   b=[5] held on stack
  ↓ recurse on a=[8]

sort([8])                                                depth 2 — BASE
  ↑ return [8]

  ↑ unwind: b1=5, main=[5,8], pend=[6(odd)]
  ↑ insert 6 (no bound) → 2 cmps → main=[5,6,8]
  ↑ return [5, 6, 8]

  ↑ unwind: sortedA=[5,6,8]
  ↑ b1=2 (partner of 5), main=[2,5,6,8]
  ↑ pend=[{3,bound=6}, {1,bound=8}]
  ↑ Jacobsthal order: insert 1 first, then 3
  ↑ insert 1 (bound=8@3): 2 cmps → main=[1,2,5,6,8]
  ↑ insert 3 (bound=6@3): 2 cmps → main=[1,2,3,5,6,8]
  ↑ return [1, 2, 3, 5, 6, 8]

```

Key Rules — Quick Reference

Rule	What it means
Only a-elements recurse	b-elements stay on the current stack frame. Their bond with the a-partner is never broken.
b ₁ is always free	The b-partner of the smallest sorted a is placed at position 0 with no comparison. It must be ≤ a ₀ by the pairing invariant.
Bound = a-partner position	Each b-element can only land LEFT of its a-partner. The binary search never looks past that point. This is the core saving over naïve insertion.
Jacobsthal order	Groups are ordered so every element in a group costs exactly $\lceil \log_2(\text{group_ceiling}) \rceil$ comparisons. Inserting high-to-low within a group prevents the bound from expanding.
findBoundPos not counted	Locating the a-partner's current position is bookkeeping — the a-partner is already sorted, so this is not a comparison between unsorted elements.
aToB is O(1)	BlockIds are consecutive integers. Using them as array indices gives direct lookup with no scan.